

Project Report: Texture-Based Image Segmentation using Laws' Filters and K-Means Clustering

Ravi Shankar Meena

Aman Raj

Arnav Gautam

April 30, 2025

Abstract

This project implements an automated image segmentation pipeline using texture analysis with Laws' filters and unsupervised K-Means clustering. The goal is to segment grayscale images, including satellite images also, into regions of similar texture. This algorithm provides a basis and used for applications in remote sensing, medical imaging, and object recognition, where different textures in images need to be identified and isolated.

1 Introduction

Segmentation of an image into regions with similar textures is an important task in various image processing fields, such as remote sensing, medical imaging, and object recognition. This project combines Laws' texture energy filters with K-Means clustering to segment images based on texture. The method helps to identify and separate different regions based on their texture, which can be useful for tasks like land classification.

This report covers the methodology and implementation of the image segmentation process, along with the results from testing the approach on several grayscale images, including satellite images.

2 Methodology

2.1 Laws' Texture Energy Filters

Laws' texture energy filters are a set of 5x5 convolution kernels designed to capture different texture patterns like level, edge, spot, wave, and ripple. These filters help identify local texture features in images. When applied to an image, each filter creates a separate feature map, resulting in 25 different maps that represent various texture patterns in the image.

2.2 Feature Extraction

The input image is first converted to grayscale. Then, each of the 25 Laws' kernels is applied to the grayscale image through convolution, highlighting different texture patterns. To reduce noise and smooth out sharp variations, a mean blur is applied to the result of each kernel using a 15x15 pixel window. This step helps in reducing high-frequency noise while keeping important texture patterns intact.

2.3 Feature Vector Formation

After obtaining the feature maps, they are combined and flattened into a 25-dimensional feature vector for each pixel in the image. These vectors represent the texture characteristics of each pixel, forming a feature space that is used for the next step of clustering.

2.4 K-Means Clustering

The next step is clustering the feature vectors using K-Means, where each pixel is assigned to one of the $k=4$ clusters. In this approach, K-Means is manually implemented by randomly selecting centroids, assigning pixels to the nearest centroid, and then updating the centroids through iterations until they stabilize. The final cluster labels for each pixel create a segmented image, where similar textures are grouped together.

2.5 Parallel Processing

To speed up processing, especially with large images or multiple files, the pipeline uses parallel processing. With the help of the joblib library, all available CPU cores are used to process the images at the same time. This

significantly reduces the processing time and makes it easier to handle multiple images in batches.

3 Implementation Details

3.1 Key Libraries Used

The following Python libraries were used for the implementation:

- **numpy**: For array manipulation and mathematical operations.
- **Pillow (PIL)**: For image reading and conversion to grayscale.
- **scipy.ndimage**: For applying convolutions and mean blur.
- **joblib**: For parallel processing and efficient batch processing of images.
- **tqdm**: For displaying progress bars during processing.

3.2 Directory Structure

The following directory structure was used for the project:

- **./images/**: Input images (in various formats, such as PNG, JPEG).
- **./output/**: Output segmented images, saved in the same format as input.

3.3 Main Steps in Code

The core steps in the image segmentation pipeline are as follows:

1. **Read and convert images to grayscale**: Images are loaded and converted to grayscale to simplify the processing.
2. **Generate Laws' 5x5 kernels**: The set of 25 Laws' texture energy filters is generated and used to convolve the image.
3. **Extract texture features**: Each grayscale image is processed using the Laws' filters to extract texture features, followed by a mean blur for noise reduction.
4. **Flatten features and cluster using K-Means**: The texture features are flattened into feature vectors, and K-Means clustering is applied to assign each pixel to a texture class.

5. **Save the segmented output image:** The final segmented image is saved with distinct texture classes for each region.

4 Results

4.1 Demonstration Images

We tested the image segmentation on various grayscale images, including satellite images. The purpose was to evaluate how well the segmentation works in separating different texture regions. The segmented results were visually compared with the original images to check if areas such as land, water, and buildings were grouped properly based on their texture features.

4.2 Example Outputs



Figure 1: Original Image



Figure 2: Segmented Image

Here we can see the segmentation of water and residential area laying near the sea.

5 Running the Streamlit Application

To make the segmentation process interactive, we developed a user-friendly web interface using Streamlit. Users can upload grayscale images, adjust the number of clusters (k), and visualize the segmented output directly in the browser.

Steps to Run the Application:

1. Ensure `app.py` and `script.py` are in the same directory.
2. Install required libraries (if not already installed):

```
pip install streamlit pillow numpy scipy joblib tqdm
```

3. Run the application using the command:

```
streamlit run app.py
```

4. Once started, it will automatically open in your default browser at:

```
http://localhost:8501
```

5. Upload a grayscale image and choose the number of clusters (k). The segmented output will be displayed on the screen.
6. **Alternatively**, if you prefer not to use the web-based interface, you can directly run the segmentation script using:

```
python3.10 script.py --k X
```

X is the k value , default is k =4. This will process all grayscale images in the `images/` folder and save the segmented outputs to the `output/` folder.

6 Conclusion

In this project, we demonstrated the effectiveness of using Laws' texture energy filters combined with K-Means clustering for texture-based image segmentation. The results show that the pipeline can accurately segment grayscale images, including satellite images, into regions with distinct textures. The use of parallel processing further improved the efficiency of the pipeline, making it scalable for processing large datasets.